Basics

Variables, types, operators

Output & Variables

`print("Hi")`	show text
`let x = 5`	constant
`var y = 3.14`	variable
`var s: String = "text"`	explicit type
`let big = 1_000_000`	readable number
`readLine()`	read input

Types

`Int  Double  Float`	numbers
`String  Character`	text
`Bool`	true / false
`Int("5")`	text to Int? (optional)
`String(5)`	anything to text
`type(of: x)`	get the type

Operators

`+  -  *  /  %`	math
`==  !=  <  >`	compare
`&&  ||  !`	logic
`1...5  1..<5`	closed / half-open range
`"Hi \(name)"`	string interpolation
`// one line  /* many */`	comments

Strings & Characters

Work with text

Basics

`s.count  s.isEmpty`	length / empty?
`s.uppercased()  s.lowercased()`	change case
`s.hasPrefix("a")  s.hasSuffix("z")`	start / end
`s.contains("a")`	contains text?
`s.replacingOccurrences(of: "a", with: "b")`	replace text
`s.trimmingCharacters(in: .whitespaces)`	trim spaces

Split & Join

`s.split(separator: ",")`	text to pieces
`parts.joined(separator: ", ")`	pieces to text
`s.components(separatedBy: " ")`	split (Foundation)
`String(repeating: "ab", count: 3)`	repeat text

Index & Convert

`s.first  s.last`	first / last char
`s[s.startIndex]`	first by index
`s.prefix(3)  s.suffix(3)`	first / last chars
`String(s.reversed())`	reverse text
`Int("42")`	text to optional Int
`"""multi-line"""`	multi-line string
`#"raw \n text"#`	raw string

Collections

Array, Dictionary, Set

Array

`var a = [1, 2, 3]`	create array
`a.append(4)`	add to end
`a.insert(0, at: 0)`	insert at index
`a.remove(at: 1)`	remove at index
`a.count  a.isEmpty`	size / empty?
`a[0]  a.first  a.last`	get items
`a.contains(2)`	has item?
`a.sorted()  a.reversed()`	new order

Dictionary

`var d = ["a": 1]`	create dictionary
`d["b"] = 2`	add / update
`d["a"]`	get value (optional)
`d["a", default: 0]`	safe get
`d.removeValue(forKey: "a")`	remove key
`d.keys  d.values`	all keys / values

Set

`var s: Set = [1, 2, 3]`	unique items
`s.insert(4)`	add item
`s.union(t)  s.intersection(t)`	set math

Collection Operations

Functional tools

Transform

`a.map { $0 * 2 }`	change each item
`a.filter { $0 > 1 }`	keep matches
`a.compactMap { Int($0) }`	map, drop nils
`a.flatMap { [$0, $0] }`	map and flatten
`a.reduce(0, +)`	fold to one value
`a.forEach { print($0) }`	run for each

Search & Check

`a.contains(3)`	has item?
`a.contains { $0 > 2 }`	any match?
`a.allSatisfy { $0 > 0 }`	all match?
`a.first { $0 > 1 }`	first match
`a.firstIndex(of: 3)`	position of item
`a.min()  a.max()`	smallest / largest

Order & Slice

`a.sorted(by: >)`	descending order
`a.sort()`	sort in place
`a.shuffled()`	random order
`a.prefix(2)  a.dropFirst(2)`	first / skip items
`a.enumerated()`	index + item
`zip(a, b)`	pair items
`Dictionary(grouping: a) { $0 % 2 }`	group by key

Control Flow

Decide and repeat

Conditions

`if x > 0 { } else { }`	branches
`guard x > 0 else { return }`	early exit
`switch x { case 1: ... default: ... }`	pattern match
`case 1...5:`	range case
`case let n where n > 5:`	case with condition
`x > 0 ? "yes" : "no"`	ternary

Loops

`for i in 0..<5 { }`	repeat 5 times
`for item in array { }`	loop items
`for (i, v) in a.enumerated() { }`	index + item
`for (k, v) in dict { }`	loop pairs
`while cond { }`	loop while true
`repeat { } while cond`	run at least once
`break  continue`	exit / skip

Functional

`a.map { $0 * 2 }`	transform each
`a.filter { $0 > 1 }`	keep matches
`a.reduce(0, +)`	fold to one value
`a.sorted { $0 > $1 }`	custom sort

Switch & Patterns

Match anything

Switch Cases

`switch x { case 1: ... }`	match a value
`case 1, 2, 3:`	several values
`case 1...5:`	a range
`case let n where n > 5:`	with condition
`case (0, 0):`	tuple match
`default:`	everything else

Binding & Tuples

`case let (a, b):`	bind tuple parts
`case (let a, 0):`	bind one part
`case .some(let v):`	unwrap optional
`case is String:`	type check
`case let s as String:`	cast and bind
`fallthrough`	continue to next case

Other Patterns

`if case .a = e { }`	one-case match
`for case let x? in opts { }`	loop non-nil values
`if x ~= 1...5 { }`	pattern match
`guard case .a = e else { return }`	match or exit

Functions

Functions and closures

Functions

`func f(a: Int) -> Int { }`	define function
`f(a: 5)`	call with label
`func f(_ a: Int)`	no label
`func f(a: Int = 1)`	default value
`func f(_ n: Int...)`	variadic
`func f(x: inout Int)`	change caller value
`func f() -> (Int, String)`	return tuple

Closures

`{ (a: Int) -> Int in a * 2 }`	closure syntax
`{ $0 * 2 }`	shorthand
`func f(done: () -> Void)`	closure param
`@escaping`	closure outlives call
`{ [weak self] in }`	avoid retain cycle

Tuples & Enums

`let t = (1, "a")`	tuple
`t.0  t.1`	read by index
`enum D { case up, down }`	enumeration
`enum E { case a(Int) }`	with value

Enums

Cases with data

Basic Enums

`enum Dir { case up, down }`	simple cases
`enum Dir: String { case up }`	raw values
`Dir.up.rawValue`	read raw value
`Dir(rawValue: "up")`	make from raw value
`enum N: Int { case a = 1 }`	int raw values
`Dir.allCases`	list cases (CaseIterable)

Associated Values

`case ok(Int)`	case with data
`case err(String, Int)`	several values
`case .ok(let n):`	read the data
`indirect enum Tree { }`	recursive enum

Members

`var label: String { switch self { } }`	computed property
`func next() -> Dir { }`	method on enum
`static let main = Dir.up`	shared value
`mutating func toggle() { }`	change self
`extension Dir { }`	add features later

Types & Protocols

struct, class, protocol

Struct & Class

`struct P { var x: Int }`	value type
`class C { var x = 0 }`	reference type
`init(x: Int) { self.x = x }`	initializer
`mutating func f()`	change struct data
`class B: A { }`	inherit
`override func f()`	replace parent
`final class C`	cannot subclass

Properties

`var area: Double { w * h }`	computed property
`lazy var data = load()`	load when needed
`var x = 0 { didSet { } }`	property observer
`static let id = 1`	type property

Protocols & Extensions

`protocol Shape { func area() }`	define protocol
`struct S: Shape { }`	conform
`extension Int { }`	add methods
`Equatable  Hashable  Codable`	common protocols

Generics & Extensions

Reuse your code

Generics

`func f<T>(_ x: T) -> T`	generic function
`struct Box<T> { var v: T }`	generic type
`func f<T: Comparable>(_ a: T)`	constrained type
`where T: Equatable`	extra rules
`some View`	opaque return type
`any Shape`	existential type

Extensions

`extension Int { }`	add to a type
`var double: Int { self * 2 }`	computed in extension
`extension Array where Element: Numeric`	conditional extension
`extension P { func f() { } }`	protocol default method

Protocols

`protocol P { func f() }`	define a contract
`associatedtype Item`	placeholder type
`protocol A: B, C { }`	combine protocols
`typealias Id = Int`	type alias

Optionals & Errors

Safe code

Optionals

`var n: Int? = nil`	optional value
`n!`	force unwrap (risky)
`if let v = n { }`	safe unwrap
`guard let v = n else { return }`	unwrap or exit
`n ?? Q`	default value
`n?.description`	optional chaining

Errors

`enum E: Error { case bad }`	error type
`func f() throws { }`	can throw
`throw E.bad`	throw error
`do { try f() } catch { }`	handle error
`try? f()  try! f()`	optional / force

Concurrency

`async  await`	async code
`Task { await f() }`	run a task
`@MainActor`	run on main thread

Codable & JSON

Save and load data

Codable

`struct U: Codable { var name: String }`	make it codable
`Codable = Encodable + Decodable`	both directions
`enum CodingKeys: String, CodingKey { }`	rename keys
`let enc = JSONEncoder()`	encoder
`let dec = JSONDecoder()`	decoder

Encode & Decode

`try enc.encode(user)`	object to Data
`try dec.decode(User.self, from: data)`	Data to object
`enc.outputFormatting = .prettyPrinted`	readable JSON
`keyDecodingStrategy = .convertFromSnakeCase`	snake_case keys
`String(data: d, encoding: .utf8)`	Data to text

Files & Defaults

`try Data(contentsOf: url)`	read a file
`try data.write(to: url)`	write a file
`UserDefaults.standard.set(1, forKey: "k")`	save small values
`UserDefaults.standard.integer(forKey: "k")`	read a value
`FileManager.default.urls(for:in:)`	documents folder

Concurrency

async, await, actors

Async / Await

`func f() async -> Int`	async function
`let v = await f()`	wait for result
`func f() async throws`	async + errors
`try await f()`	call async throwing
`Task { await f() }`	start async work
`Task.sleep(for: .seconds(1))`	pause

Parallel Work

`async let a = f()`	start in parallel
`await withTaskGroup(of: Int.self) { }`	many tasks
`withThrowingTaskGroup(of: Int.self) { }`	many, may fail
`task.cancel()`	cancel a task
`Task.isCancelled`	was it cancelled?

Actors & Main

`actor Counter { var n = 0 }`	safe shared state
`await counter.n`	access an actor
`@MainActor`	run on main thread
`Sendable`	safe across threads
`DispatchQueue.main.async { }`	old main-thread call

Memory & Access

ARC and visibility

ARC

`class A { }`	reference type (ARC)
`struct S { }`	value type (copied)
`weak var d: A?`	does not keep alive
`unowned let o: A`	never nil reference
`[weak self] in`	avoid closure cycles
`deinit { }`	runs when freed

Access Control

`private`	this scope only
`fileprivate`	this file only
`internal`	this module (default)
`public`	other modules can use
`open`	public + subclassable
`private(set) var x`	read-only outside

Value Semantics

`var b = a`	struct: copies
`mutating func f()`	struct changes self
`===  !==`	same object? (classes)
`lazy var x = f()`	create on first use
`@inline(__always)`	hint to the compiler

SwiftUI & Tips

UI basics and habits

SwiftUI

`struct V: View { body }`	a view
`Text("Hi")`	label
`VStack  HStack  ZStack`	layout stacks
`Button("Tap") { }`	button
`@State var n = 0`	local state
`@Binding var n: Int`	shared state
`.padding() .font(.title)`	modifiers
`List { } NavigationStack { }`	list / navigation

Good Habits

`Prefer let over var`	safer code
`Avoid force unwrap !`	no crashes
`Use guard for early exit`	clean flow

Keep coding. Keep learning.

Made by

@hafizibrahimsuriya

Save this sheet | Share it | Practice daily